

**MESIN PEMBELAJARAN (*Machine Learning*)**  
**PEMBAHASAN *UNSUPERVISED LEARNING* DAN *PREPROCESSING***



**Disusun Oleh :**

Nama : Aulia Afdhal Hafidzah  
NPM : G1A024029  
Kelas : A

**Dosen Pengampu :**

Ir. Arie Vatesia, S.T., M.T.i., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS BENGKULU**  
**2026**

## A. Tantangan Unsupervised Learning

Tantangan utama dalam *unsupervised learning* adalah mengevaluasi apakah algoritma benar-benar mempelajari sesuatu yang berguna. Karena data umumnya tidak memiliki label, kita tidak mengetahui output yang benar sehingga sulit menyatakan bahwa model berhasil. Contohnya, algoritma *clustering* bisa saja mengelompokkan foto wajah berdasarkan sudut pengambilan gambar (wajah tampak depan berbanding profil) alih-alih berdasarkan identitas orang. Hasil semacam ini sah secara teknis, tetapi bukan yang kita inginkan, dan tidak ada cara untuk memberi tahu algoritma apa yang kita cari. Evaluasi hasil algoritma *unsupervised* sering kali hanya dapat dilakukan lewat inspeksi manual. Akibatnya, algoritma *unsupervised* lazim digunakan dalam konteks eksplorasi ketika ilmuwan data ingin memahami data lebih dalam, atau sebagai langkah praproses bagi algoritma *supervised*. Representasi baru hasil transformasi kadang meningkatkan akurasi model *supervised*, atau mengurangi konsumsi memori dan waktu komputasi. Menariknya, metode praproses dan *scaling* yang dibahas di awal bab ini sendiri termasuk *unsupervised*, karena tidak memanfaatkan informasi label sama sekali.

## B. Preprocessing dan Scaling

### 1. Jenis - Jenis Scaling

- a. *StandardScaler* memastikan setiap fitur memiliki rata-rata (*mean*) nol dan varians satu, sehingga semua fitur berada pada magnitudo yang sama, meskipun metode ini tidak menjamin nilai minimum dan maksimum tertentu.
- b. *RobustScaler* bekerja serupa tetapi menggunakan median dan kuartil alih-alih rata-rata dan varians, sehingga mengabaikan titik data yang menyimpang jauh (*outliers*) seperti kesalahan pengukuran yang dapat merusak metode *scaling* lainnya.
- c. *MinMaxScaler* menggeser data sehingga semua fitur berada tepat pada rentang nol sampai satu.

- d. *Normalizer* melakukan penskalaan yang berbeda sifatnya: setiap titik data diskalakan sehingga vektor fiturnya memiliki panjang *Euclidean* satu, artinya setiap titik diproyeksikan ke lingkaran (atau bola hipersfer pada dimensi lebih tinggi) berjari-jari satu. Normalisasi ini dipakai ketika hanya arah vektor yang penting, bukan panjangnya.
2. Penerapan Transformasi pada Dataset Kanker Payudara

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, random_state=1)
scaler = MinMaxScaler()
scaler.fit(X_train)          # hitung min & max tiap fitur
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Kode di atas menunjukkan pola antarmuka *scikit-learn*: objek *scaler* dibuat, lalu metode *fit* dipanggil pada data latih untuk menghitung nilai minimum dan maksimum setiap fitur, kemudian metode *transform* dipanggil untuk menerapkan transformasi. Perhatikan bahwa *scaler* hanya menerima data X, tanpa label y, yang menegaskan sifat *unsupervised*-nya. Data yang telah ditransformasi berbentuk sama dengan data asli, tetapi kini seluruh fitur data latih berada pada rentang nol sampai satu. Hal penting muncul ketika mentransformasi data uji: nilai minimum dan maksimum data uji setelah *scaling* tidak tepat nol dan satu, bahkan ada fitur yang keluar dari rentang tersebut. Penjelasanannya adalah *MinMaxScaler* (dan semua *scaler* lainnya) selalu menerapkan transformasi yang persis sama pada data latih dan data uji, yaitu mengurangi minimum data latih dan membagi dengan rentang data latih, yang tentu berbeda dari statistik data uji. Inilah perilaku yang benar.

Visualisasi tiga panel pada gambar buku menunjukkan bahwa menskalakan data latih dan uji secara terpisah mengubah susunan relatif

data secara acak; titik uji bergeser tidak konsisten terhadap titik latih. Sebagai analogi, jika data uji hanya berupa satu titik, tidak mungkin menskalakannya dengan benar memenuhi syarat minimum dan maksimum; ukuran data uji tidak seharusnya mengubah cara pemrosesan.

### 3. Shortcut `fit_transform`

Karena tugas menggabungkan *fit* lalu *transform* sangat umum, seluruh model yang memiliki metode *transform* juga menyediakan metode *fit\_transform* yang lebih efisien secara komputasi.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit(X).transform(X) # dua langkah berantai
X_scaled_d = scaler.fit_transform(X) # satu langkah, lebih efisien
```

### 4. Pengaruh Scaling terhadap Supervised Learning

Dampak *scaling* sangat nyata pada performa SVM di dataset kanker payudara. Tanpa *scaling*, SVM dengan  $C=100$  hanya mencapai akurasi uji 0.63. Setelah data diskalakan dengan *MinMaxScaler*, akurasi melonjak menjadi 0.97, dan dengan *StandardScaler* menjadi 0.96. Meskipun *scaling* tidak melibatkan matematika yang rumit, buku menyarankan memakai mekanisme bawaan *scikit-learn* alih-alih menulis sendiri, karena kesalahan mudah terjadi bahkan pada perhitungan sederhana. Semua kelas praproses berbagi antarmuka *fit* dan *transform* yang sama, sehingga mengganti satu metode dengan metode lain cukup dengan mengganti kelasnya.

## C. Dimensionality Reduction, Feature Extraction, dan Manifold Learning

*Principal Component Analysis* (PCA) sebagai algoritma paling sederhana dan paling luas dipakai, *Non-Negative Matrix Factorization* (NMF) untuk ekstraksi fitur, serta t-SNE untuk visualisasi.

### 1. Principal Component Analysis (PCA)

PCA memutar dataset sedemikian rupa sehingga fitur hasil rotasi tidak berkorelasi secara statistik. Algoritma pertama-tama mencari arah varians maksimum, yang disebut komponen utama pertama; inilah arah yang

memuat informasi terbanyak atau arah tempat fitur saling berkorelasi paling kuat. Kemudian dicari arah yang memuat informasi terbanyak sambil ortogonal (tegak lurus) terhadap arah pertama. Arah-arrah yang ditemukan inilah yang disebut komponen utama (*principal components*), dan secara umum jumlahnya sebanyak fitur asli. Rotasi biasanya diikuti pemilihan sebagian komponen saja sesuai kepentingannya dalam menjelaskan data, sehingga terjadi reduksi dimensi. Sebelum rotasi, rata-rata data dikurangkan sehingga data terpusat di nol. PCA juga dapat dipakai untuk menghilangkan efek derau dengan memutar data kembali ke ruang asli setelah membuang komponen bernilai rendah.

## 2. Penerapan PCA untuk Visualisasi Dataset Kanker Payudara

Salah satu penerapan PCA yang paling umum adalah visualisasi data berdimensi tinggi. Dataset kanker payudara memiliki 30 fitur yang jika dipasangkan menghasilkan 420 diagram pencar; mustahil diamati semuanya. Histogram per fitur per kelas (ganas dan jinak) memberi gambaran distribusi, misalnya fitur *smoothness error* tampak tidak informatif karena kedua histogram tumpang tindih, sementara *worst concave points* tampak sangat informatif karena keduanya terpisah, tetapi histogram tidak memperlihatkan interaksi antar variabel. PCA dapat menangkap interaksi utama tersebut.

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

cancer = load_breast_cancer()
scaler = StandardScaler()
scaler.fit(cancer.data)
X_scaled = scaler.transform(cancer.data)
pca = PCA(n_components=2)    # simpan dua komponen utama
pca.fit(X_scaled)
X_pca = pca.transform(X_scaled)
print("Original shape:", X_scaled.shape) # (569, 30)
print("Reduced shape:", X_pca.shape)     # (569, 2)
```

Sebelum menerapkan PCA, data diskalakan dengan *StandardScaler* agar setiap fitur memiliki varians satu. Data berdimensi 30 lalu direduksi menjadi 2 dimensi. Diagram pencar dua komponen pertama, yang diberi warna berdasarkan kelas, memperlihatkan kedua kelas terpisah cukup baik. Penting dicatat bahwa PCA adalah metode *unsupervised* dan tidak memakai informasi kelas saat mencari rotasi; ia hanya melihat korelasi data. Pemisahan yang tampak mengindikasikan pengklasifikasi linear pun dapat bekerja baik di ruang baru ini. Titik kelas ganas juga terlihat lebih menyebar daripada kelas jinak.

Kelemahan PCA adalah kedua sumbu hasil transformasi sulit diinterpretasikan karena setiap komponen merupakan kombinasi kompleks dari seluruh fitur asli. Komponen disimpan pada atribut *components\_* dengan bentuk (2, 30); setiap baris satu komponen yang terurut menurut kepentingannya, setiap kolom satu fitur asli. Peta panas (*heat map*) koefisien menunjukkan komponen pertama bertanda sama pada semua fitur, artinya terdapat korelasi umum antar seluruh fitur, sedangkan komponen kedua bertanda campuran.

```
print("PCA component shape:", pca.components_.shape) # (2, 30)
plt.matshow(pca.components_, cmap="viridis")
plt.yticks([0, 1], ["First component", "Second component"])
plt.colorbar()
plt.xticks(range(len(cancer.feature_names)),
cancer.feature_names, rotation=60, ha="left")
```

### 3. Eigenfaces untuk Feature Extraction

Ide ekstraksi fitur adalah menemukan representasi data yang lebih sesuai untuk analisis daripada representasi mentah. Contoh klasiknya adalah citra wajah, yang tersusun dari ribuan piksel merah, hijau, dan biru yang hanya bermakna apabila diperhatikan bersama. Buku menggunakan dataset *Labeled Faces in the Wild* (LFW) berisi wajah selebritas dari internet, dengan 3.023 citra berukuran 87 kali 65 piksel milik 62 orang. Dataset ini timpang; George W. Bush memiliki 530 citra dan Colin Powell

236, sehingga hanya maksimal 50 citra per orang yang diambil, dan nilai piksel dinormalisasi ke rentang nol sampai satu agar stabil secara numerik.

```
from sklearn.datasets import fetch_lfw_people
people = fetch_lfw_people(min_faces_per_person=20, resize=0.7)
mask = np.zeros(people.target.shape, dtype=bool)
for target in np.unique(people.target):
    mask[np.where(people.target == target)[0][:50]] = 1
X_people = people.data[mask] / 255.
y_people = people.target[mask]
```

Tugas pengenalan wajah lazim diselesaikan dengan *one-nearest-neighbor classifier*, karena setiap orang hanya memiliki sedikit citra latih. Tanpa PCA, akurasi uji hanya 0.27, sebenarnya tidak buruk untuk 62 kelas (tebakan acak sekitar 1.5 persen) tetapi juga tidak bagus. Masalahnya, jarak pada ruang piksel mentah adalah ukuran kemiripan wajah yang buruk; menggeser wajah satu piksel ke kanan saja menghasilkan representasi yang berbeda drastis, padahal manusia menilai kemiripan lewat usia, gender, ekspresi, atau gaya rambut. Solusinya adalah PCA dengan opsi *whitening*, yang menskalakan ulang komponen utama agar berskala sama, ibarat data hasil rotasi berbentuk lingkaran alih-alih elips. Dengan 100 komponen utama, akurasi meningkat signifikan dari 0.27 menjadi 0.36, mengonfirmasi bahwa komponen utama memberikan representasi yang lebih baik.

```
pca = PCA(n_components=100, whiten=True,
random_state=0).fit(X_train)
X_train_pca = pca.transform(X_train) # (1537, 100)
X_test_pca = pca.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train_pca, y_train)
print("Test set accuracy: {:.2f}".format(knn.score(X_test_pca, y_test)))
```

Komponen PCA pada data citra dapat divisualisasikan sebagai citra berukuran sama dengan input. Komponen pertama tampak mengodekan kontras antara wajah dan latar, komponen kedua perbedaan pencahayaan

antara sisi kanan dan kiri wajah, dan seterusnya. Interpretasi lain: titik data dapat dinyatakan sebagai penjumlahan berbobot komponen utama, dan rekonstruksi citra dengan 10, 50, 100, 500, hingga 2000 komponen memperlihatkan bahwa makin banyak komponen, makin banyak detail yang terjaga; dengan komponen sebanyak piksel, rekonstruksi menjadi sempurna. Metode *inverse\_transform* dipakai untuk memutar data kembali ke ruang asli. Namun, scatter plot seluruh wajah dengan dua komponen pertama hanya menghasilkan satu gumpalan besar tanpa pemisahan kelas, karena dua komponen pertama hanya menangkap karakteristik kasar wajah.

#### 4. Non-Negative Matrix Factorization (NMF)

NMF adalah algoritma *unsupervised* lain untuk ekstraksi fitur yang bekerja serupa dengan PCA dan juga dapat dipakai untuk reduksi dimensi. Sama seperti PCA, setiap titik data dinyatakan sebagai penjumlahan berbobot komponen. Bedanya, NMF mengharuskan komponen maupun koefisien bernilai non-negatif, sehingga metode ini hanya dapat diterapkan pada data yang fiturnya non-negatif, karena penjumlahan non-negatif dari komponen non-negatif tak mungkin menghasilkan nilai negatif.

Dekomposisi non-negatif sangat membantu untuk data yang terbentuk dari penjumlahan beberapa sumber independen, seperti rekaman audio berisi banyak orang berbicara atau musik dengan banyak instrumen. Secara umum NMF menghasilkan komponen yang lebih mudah diinterpretasikan daripada PCA, karena komponen dan koefisien negatif dapat menimbulkan efek saling menghapus yang membingungkan, sementara tanda pada komponen PCA sebenarnya arbitrer.

Berbeda dengan PCA, posisi data relatif terhadap titik asal (0, 0) penting bagi NMF; komponen non-negatif dapat dipandang sebagai arah dari titik asal menuju data. Pada data sintetis dua dimensi dengan dua komponen, seluruh titik dapat ditulis sebagai kombinasi positif kedua komponen. Jika hanya satu komponen, NMF menciptakan komponen yang menunjuk ke rata-rata data karena arah itu paling baik menjelaskan data. Konsekuensinya, mengurangi jumlah komponen pada NMF tidak sekadar

membuang beberapa arah, tetapi menciptakan sekumpulan komponen yang sama sekali berbeda. Komponen NMF juga tidak terurut; tidak ada komponen pertama, semua berperan setara. NMF memakai inisialisasi acak sehingga hasil bisa berbeda tergantung benih acak. Pada dataset wajah, komponen NMF dengan 15 komponen semuanya positif sehingga menyerupai prototipe wajah jauh lebih baik daripada komponen PCA; misalnya komponen ketiga menampilkan wajah agak menoleh ke kanan dan komponen ketujuh menoleh ke kiri. Citra dengan koefisien tinggi pada komponen tersebut memang menunjukkan pola sesuai. Kualitas rekonstruksi NMF mirip PCA tetapi sedikit lebih buruk, dan memang NMF lazimnya dipakai untuk menemukan pola menarik, bukan untuk mengode atau merekonstruksi data. Contoh penerapan yang elegan adalah pemulihan sinyal. Tiga sinyal sumber dicampur menjadi 100 kanal pengukuran melalui matriks campuran acak, menghasilkan data berbentuk (2000, 100). NMF dengan tiga komponen berhasil menemukan kembali ketiga sinyal asli dengan cukup baik, sementara PCA gagal dan memakai komponen pertamanya untuk menjelaskan mayoritas variasi. Urutan komponen NMF yang kebetulan sama dengan sinyal asli murni kebetulan, karena komponen NMF tidak punya urutan alami.

```
S = mglearn.datasets.make_signals()      # sinyal asli
A = np.random.RandomState(0).uniform(size=(100, 3))
X = np.dot(S, A.T)                      # campuran (2000, 100)
nmf = NMF(n_components=3, random_state=42)
S_ = nmf.fit_transform(X)               # (2000, 3)
pca = PCA(n_components=3)
H = pca.fit_transform(X)
```

## 5. Manifold Learning dengan t-SNE

Sifat PCA yang hanya memutar lalu membuang arah membatasi kegunaannya untuk visualisasi, seperti terlihat pada scatter plot dataset wajah. Kelas algoritma bernama *manifold learning* memungkinkan pemetaan jauh lebih kompleks dan sering menghasilkan visualisasi lebih baik. Algoritma ini memang ditujukan untuk visualisasi sehingga jarang

menghasilkan lebih dari dua fitur baru. Beberapa di antaranya, termasuk t-SNE, hanya dapat mentransformasi data latihnya dan tidak mendukung transformasi data baru, sehingga tidak dapat diterapkan pada data uji. Karena itu *manifold learning* berguna untuk analisis data eksploratif tetapi jarang dipakai bila tujuan akhirnya adalah *supervised learning*. t-SNE berusaha menemukan representasi dua dimensi yang mempertahankan jarak antar titik sebaik mungkin, dengan penekanan lebih pada titik-titik yang berdekatan; ia mencoba mempertahankan informasi tentang titik mana yang bertetangga. t-SNE diuji pada dataset digit tulisan tangan bawaan *scikit-learn* berupa citra grayscale 8 kali 8 untuk digit 0 sampai 9. Perbandingannya mencolok: dengan dua komponen pertama PCA, digit 0, 6, dan 4 terpisah cukup baik tetapi sebagian besar digit lain tumpang tindih; dengan t-SNE, seluruh kelas terpisah dengan jelas meskipun metode ini sepenuhnya tidak mengetahui label kelas. Perlu dicatat, kelas TSNE tidak memiliki metode *transform* sehingga digunakan *fit\_transform*. Parameter *perplexity* dan *early\_exaggeration* dapat diatur, tetapi dampaknya biasanya kecil dan pengaturan bawaan sering sudah baik.

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
digits = load_digits()
pca = PCA(n_components=2)
pca.fit(digits.data)
digits_pca = pca.transform(digits.data)
tsne = TSNE(random_state=42)
digits_tsne = tsne.fit_transform(digits.data) # tanpa metode transform
```

#### D. Clustering

*Clustering* adalah tugas mempartisi dataset menjadi kelompok-kelompok yang disebut klaster, dengan tujuan titik dalam satu klaster saling mirip dan titik antar klaster berbeda. Serupa dengan klasifikasi, algoritma *clustering* menetapkan nomor klaster pada setiap titik, tetapi tidak ada

kebenaran dasar (*ground truth*) sehingga label itu sendiri tidak punya makna sebelumnya (*a priori*).

## 1. k-Means Clustering

*k-means* adalah algoritma *clustering* paling sederhana dan paling umum. Ia berusaha menemukan pusat-pusat klaster yang merepresentasikan wilayah data, dengan bergantian antara dua langkah: menetapkan setiap titik ke pusat klaster terdekat, lalu menetapkan setiap pusat sebagai rata-rata titik-titik yang ditetapkan padanya. Algoritma berhenti ketika penugasan titik tidak lagi berubah.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X, y = make_blobs(random_state=1)
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
print(kmeans.labels_) # label klaster data latih
print(kmeans.predict(X)) # hasilnya sama dengan labels_
```

Label klaster ditemukan pada atribut *labels\_*, dan titik baru dapat ditetapkan lewat metode *predict* yang menentukannya ke pusat terdekat tanpa mengubah model. Buku menekankan bahwa penomoran klaster bersifat arbitrer karena inisialisasi acak; informasi satu-satunya adalah bahwa semua titik berlabel sama saling mirip. Pusat klaster tersimpan di *cluster\_centers\_*. Jika parameter *n\_clusters* tidak diberikan, nilainya delapan tanpa alasan khusus.

## 2. Kegagalan k-Means

Bahkan dengan jumlah klaster yang benar, *k-means* bisa gagal memulihkannya. Setiap klaster didefinisikan semata-mata oleh pusatnya sehingga berbentuk cembung (*convex*); *k-means* hanya mampu menangkap bentuk sederhana. Ia mengasumsikan semua klaster berdiameter sama, selalu menarik batas tepat di tengah antar pusat, dan mengasumsikan semua arah sama pentingnya. Akibatnya, pada data dengan kepadatan klaster berbeda, sebagian titik jauh terseret ke klaster lain; pada data yang diregangkan diagonal, *k-means* gagal; dan pada data

*two moons* berbentuk dua bulan sabit, ia sama sekali tidak menemukan bentuk yang diharapkan.

### 3. Vector Quantization: k-Means sebagai Dekomposisi

Meskipun algoritma *clustering*, *k-means* punya kesejajaran menarik dengan metode dekomposisi. PCA mencari arah varians maksimum, NMF mencari komponen aditif, sedangkan *k-means* merepresentasikan setiap titik dengan satu pusat kluster. Pandangan ini disebut *vector quantization*. Perbandingan pada data wajah dengan 100 komponen memperlihatkan pusat kluster *k-means* berperan seperti komponen PCA dan NMF, dengan rekonstruksi berupa pusat kluster terdekat.

```
kmeans = KMeans(n_clusters=100, random_state=0).fit(X_train)
X_reconstructed_pca = pca.inverse_transform(pca.transform(X_test))
X_reconstructed_kmeans = kmeans.cluster_centers_[kmeans.predict(X_test)]
X_reconstructed_nmf = np.dot(nmf.transform(X_test), nmf.components_)
```

Keistimewaan *vector quantization* adalah boleh memakai lebih banyak kluster daripada dimensi input. Pada data *two moons* yang hanya dua dimensi, PCA atau NMF tak berdaya, tetapi sepuluh pusat kluster *k-means* menghasilkan representasi sepuluh dimensi yang memungkinkan model linear memisahkan kedua bulan sabit. Representasi bisa lebih ekspresif lagi dengan memakai jarak ke setiap pusat kluster sebagai fitur melalui metode *transform*, menghasilkan data berbentuk (200, 10). Kelebihan *k-means* adalah mudah dipahami, cepat, dan mudah diskalakan; *scikit-learn* menyediakan *MiniBatchKMeans* untuk dataset sangat besar. *scikit-learn* menjalankan algoritma sepuluh kali dengan sepuluh inisialisasi acak dan mengembalikan hasil terbaik, yakni yang jumlah varians klasternya paling kecil. Kekurangannya: bergantung pada benih acak, membuat asumsi bentuk kluster yang ketat, dan menuntut kita menentukan jumlah kluster yang di dunia nyata sering tidak diketahui.